

Reviewer Pack — Minimal Order of Geometric Flows and Boolean Function Entrop...

Entry cced13a3c0cc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 21:13:25 UTC

Minimal Order of Geometric Flows and Boolean Function Entropy

Entry ID: cced13a3c0cc **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 21:13:25 UTC

1. Conjecture

Field A (mathematical branch): Geometric Flow Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

The minimal order of the geometric flow for a given boolean function is linearly correlated with its entropy, such that $O(GF(f)) = \Theta(\text{Entropy}(f))$. For all boolean functions f , the minimal order of their geometric flow satisfies $O(GF(f)) = \Theta(H(f))$, where $H(f)$ denotes the Shannon entropy of f .

Rationale (proposer's reasoning):

Geometric flows provide a dynamical system perspective on structures like boolean functions. This conjecture suggests that the complexity and structure of these flows could reveal insight into the intrinsic properties of boolean functions, which are central to circuit complexity theory. The correlation between geometric flow order and entropy could expose non-trivial structural information.

Taxonomy category: geometric_flow_entropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9c9655b1d13e5860

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all boolean functions f , the ratio of the minimal geometric flow order ($O(GF(f))$) to the Shannon entropy ($H(f)$) of f is within a constant factor C , where $C \geq 1$ and the ratio is computed across multiple seeds. The criterion is falsified if any seed produces a ratio exceeding C .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:geometric flow AND boolean function entropy - shannon entropy in boolean circuits AND geometric flows - entropy of boolean functions AND minimal order geometric flow

Top relevant hits considered: - [http://arxiv.org/abs/1612.05917v1] Linear Quantum Entropy and Non-Hermitian Hamiltonians - [http://arxiv.org/abs/1407.5475v1] Control Volume Analysis, Entropy Balance and the Entropy Production in Flow Systems - [http://arxiv.org/abs/1506.08519v1] Information flow and entropy production on Bayesian networks - [http://arxiv.org/abs/2107.09581v2] Entropy as a Topological Operad Derivation - [http://arxiv.org/abs/quant-ph/0305062v2] Renyi extrapolation of Shannon entropy - [http://arxiv.org/abs/quant-ph/0408192v5] Differential entropy and time - [http://arxiv.org/abs/0903.3848v1] Join-irreducible Boolean functions - [http://arxiv.org/abs/0808.0684v1] 9-variable Boolean Functions with Nonlinearity 242 in the Generalized Rotation Class

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from itertools import product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def shannon_entropy(f):
        counts = [f.count(i) for i in range(2)]
        total = sum(counts)
        if total == 0:
            return 0
        probabilities = [count / total for count in counts]
        entropy = -sum(p * math.log2(p) for p in probabilities if p > 0)
        return entropy

    def geometric_flow_order(f):
        n = len(f)
        order = 0
        while True:
```

```

        new_f = ''.join(str(1 - int(bit)) for bit in f)
        if new_f == f:
            break
        f = new_f
        order += 1
    return order

def generate_boolean_function(n):
    return ''.join(random.choice('01') for _ in range(n))

instances_tested = 0
total_order = 0
total_entropy = 0
n_max = 0

for n in [5, 10, 15, 20, 30, 40]:
    if n > 30:
        break
    for _ in range(5):
        f = generate_boolean_function(n)
        instances_tested += 1
        n_max = max(n_max, n)
        order = geometric_flow_order(f)
        entropy = shannon_entropy(f)
        total_order += order
        total_entropy += entropy

if instances_tested < 30:
    return {
        "metric_name": "Order/Entropy Ratio",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

ratio = total_order / total_entropy
return {
    "metric_name": "Order/Entropy Ratio",
    "metric_value": ratio,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": True if 1 <= ratio <= 2 else False,
    "counterexample": "" if 1 <= ratio <= 2 else f"Ratio out of bounds: {ratio}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

if all(r["conjecture_holds"] for r in results):
    mean_ratio = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = 1.0
    print(f"RESULT: SUPPORTED mean={mean_ratio} std=0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot verify the conjecture's support or falsification. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13762
2	preregistration	ollama_remote	glm4:latest	0	0	9530
3	novelty	ollama_remote	glm4:latest	0	0	8176
4	novelty	ollama_remote	glm4:latest	0	0	13744
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23390
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8165
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7975
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8711
9	judge	ollama_remote	glm4:latest	0	0	34195

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 127648 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/cced13a3c0cc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cced13a3c0cc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cced13a3c0cc.tar.gz` (if generated)